

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«Алгоритми та структури даних»

ЗВІТ
з практичної роботи №3

Виконав:
Студент групи КН-25-1
Гнатюк Є.М.
Перевірила:
Доцент кафедри КІЕ
Сидоренко В.М.

Кременчук 2026

Тема: Алгоритми сортування та їх складність. Порівняння алгоритмів сортування

Мета: опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

Завдання 1: Вивчити самостійно і записати (будь-яким способом) алгоритм бульбашкового сортування. Оцінити асимптотику алгоритму сортування методом бульбашки в найгіршому і в найкращому випадку. Порівняти за цими показниками бульбашковий алгоритм з алгоритмом сортування вставленням. Чому на практиці бульбашковий алгоритм виявляється менш ефективним у порівнянні з сортуванням методом зливанням?

1. Оцінка асимптотики сортування бульбашкою:

- **Найгірший випадок:** $T(n^2)$. Виникає на масиві, відсортованому навпаки. Алгоритм робить повну кількість порівнянь і перестановок для кожної пари.
- **Найкращий випадок:** $T(n)$. Виникає на вже відсортованому масиві. Завдяки прапорцю перестановок алгоритм робить лише один прохід ($n-1$ порівнянь) і завершує роботу.

2. Порівняння з сортуванням вставленням: Обидва алгоритми мають однакову теорію: $T(n)$ у найкращому та $T(n^2)$ у найгіршому випадку. Проте на практиці сортування вставленням працює швидше, оскільки робить значно менше порівнянь та операцій запису в пам'ять, ніж постійні покрокові обміни «бульбашки».

3. Чому «бульбашка» менш ефективна за сортування злиттям: Сортування злиттям використовує стратегію «розділяй і володарюй», гарантуючи стабільну швидкість $T(n * \log n)$ у будь-якому випадку. При збільшенні обсягу даних квадратичний темп росту $T(n^2)$ росте катастрофічно швидше, ніж лінійно-логарифмічний $T(n * \log n)$.

Завдання 2: Оцінити асимптотичну складність алгоритму сортування зливанням, скориставшись основною теоремою рекурсії.

За парадигмою «розділяй і володарюй» для сортування зливанням:

1. Задача ділиться на $a = 2$ підзадачі.
2. Розмір кожної підзадачі зменшується в $b = 2$ рази (ділення навпіл).
3. Складність процедури злиття є лінійною, тобто $O(n)$, що дає степінь $d = 1$.

Рекурентне співвідношення за основною теоремою рекурсії: $T(n) = 2 * T(n/2) + O(n^1)$

Перевіряємо умову теореми: $\log_b(a) = \log_2(2) = 1$

Оскільки $d = \log_b(a)$ (значення $1 = 1$), ми потрапляємо під другий випадок основної теореми рекурсії.

Висновок: За формулою теореми складність становить $O(n^d * \log n)$. Підставивши $d = 1$, отримуємо підсумкову асимптотичну складність: **$O(n * \log n)$** .

Завдання 3: Оцінити асимптотичну складність алгоритму швидкого сортування, скориставшись основною теоремою рекурсії.

1. Найкращий випадок: Виникає, коли опорний елемент ділить масив точно навпіл.

- Кількість підзадач: $a = 2$.
- Зменшення розміру підзадач: $b = 2$.
- Складність розділення масиву: $O(n^1)$, тобто $d = 1$.

Рівняння має вигляд: $T(n) = 2 * T(n/2) + O(n)$. Оскільки $\log_2(2) = d = 1$, за другим випадком теореми складність становить **$\Theta(n * \log n)$** .

2. Найгірший випадок: Виникає, коли масив уже відсортований, а як пілот обирається крайній елемент. Масив ділиться на підзадачі розміром 0 та $n-1$ елементів. Основна теорема рекурсії **не застосовується**, бо розмір підзадачі не зменшується в " b " разів (немає ділення на константу $b > 1$). Складність шукається через суму арифметичної прогресії: $T(n) = T(n-1) + O(n) = n * (n+1) / 2$. Це дає квадратичну асимптотику **$\Theta(n^2)$** .

Контрольні відповіді на питання

1. Що таке асимптотична складність алгоритму сортування і чому вона важлива для порівняння алгоритмів? Це метрика, яка показує залежність кількості операцій від розміру масиву при прагненні n до нескінченності. Вона важлива, бо дозволяє оцінити та порівняти ефективність алгоритмів на великих обсягах даних без прив'язки до потужності ПК.

2. Які алгоритми сортування мають квадратичну складність у найгіршому випадку? Поясніть, чому це може бути проблемою для великих обсягів даних. Квадратичну складність $\Theta(n^2)$ у найгіршому випадку мають сортування вибором, вставлянням, бульбашкою та швидке сортування. Це проблема для

великих даних, адже при збільшенні масиву в 10 разів час роботи зростає у 100 разів, викликаючи зависання програми.

3. В чому полягає перевага сортування злиттям над сортуванням вставками для великих наборів даних? Сортування злиттям гарантує швидкість $O(n \cdot \log n)$ навіть у найгіршому випадку, тоді як сортування вставками падає до $O(n^2)$. Логарифмічний темп росту на великих масивах працює в рази швидше за квадратичний.

4. Які алгоритми сортування використовуються для сортування списків у стандартних бібліотеках мов програмування, таких як Python, Java або C++?

- **Python:** Timsort (гібрид сортування вставлянням та злиттям).
- **Java:** Timsort для об'єктів та Dual-Pivot QuickSort для примітивних типів.
- **C++:** Introsort (гібрид швидкого, пірамідального та сортування вставлянням).

5. Яка різниця між алгоритмами сортування злиттям і швидким сортуванням? У яких випадках краще використовувати кожен з цих алгоритмів?

- **Злиттям:** Працює за стабільні $O(n \cdot \log n)$, але потребує додаткову пам'ять для копій масиву. Краще для зв'язаних списків або коли важлива гарантія часу.
- **Швидке:** Сортує "на місці" (без додаткової пам'яті) і в середньому швидше за злиття, але в найгіршому випадку може впасти до $O(n^2)$. Краще для стандартних масивів при обмеженій ОЗП.

6. Які фактори слід враховувати при виборі алгоритму сортування для конкретної задачі? Розмір вхідного масиву, ступінь його початкової відсортованості, обмеження по пам'яті ("на місці" чи ні), стабільність алгоритму та швидкість доступу до носія даних.